

The three **subtests** modes of **SVP()**

14 September 2026

Version 0.2.0 accepts three values of **subtests**: "**none**", "**right**", and "**both**". The former "**left**" label was removed because, in the incremental **SVP()** implementation, it executed exactly the same inclusive-left branch as "**both**". It is also rejected by **svp0()** so that the two public interfaces have the same choices. This change does not concern the internal left-hand geometric states used by **FOCuS**.

Common recursion

Let $y_{s..t} = (y_{s+1}, \dots, y_t)$, where s is a zero-based boundary and t is the one-based inclusive endpoint. Write

$$\mathcal{C}(s, t) = \sum_{i=s+1}^t (y_i - \bar{y}_{s..t})^2 \quad \text{and} \quad V_\gamma(s, t) = \mathbf{1}\{y_{s..t} \text{ is valid}\}.$$

Singletons are always valid. For longer segments, the compiled test must be finite and strictly below **gamma**. Both public routines solve the lexicographic problem

$$\hat{\tau} \in \arg \min_{\substack{0=\tau_0 < \dots < \tau_K=n \\ V_\gamma(\tau_{j-1}, \tau_j)=1, \ j=1, \dots, K}} \left(K, \sum_{j=1}^K \mathcal{C}(\tau_{j-1}, \tau_j) \right). \quad (1)$$

With $R_t = (K_t, Q_t)$ and $R_0 = (0, 0)$, the unpruned recursion is

$$R_t = \min_{\substack{0 \leq s < t \\ V_\gamma(s, t)=1}} \left\{ R_s + (1, \mathcal{C}(s, t)) \right\}. \quad (2)$$

Thus K_t is minimized first; Q_t breaks ties among partitions having the same K_t . Candidate boundaries are compared in increasing order and the C++ cost comparison is strict. Consequently, an exact floating-point tie retains the smallest visited boundary. Backtracking the predecessor column of **R** from n gives **changepoints**, including the final endpoint n .

Every active boundary s in **SVP()** owns an incremental validity-test state, the last observation fed to it, and its current validity. A deferred state is caught up by feeding each missing observation once. Ordinary catch-up checks validity only at the requested endpoint; the special right-pruning replay checks every missed endpoint. Centered first and second prefix sums give $\mathcal{C}(s, t)$ in $O(1)$ while avoiding translation-dependent numerical cancellation.

Pruning assumptions and three modes

Put $I_\gamma(s, t) = 1 - V_\gamma(s, t)$. The right-extension implication is

$$I_\gamma(s, t) = 1 \implies I_\gamma(s, u) = 1, \quad u > t, \quad (3)$$

and the left-extension implication is

$$I_\gamma(s, t) = 1 \implies I_\gamma(r, t) = 1, \quad r < s. \quad (4)$$

These are sufficient conditions for the corresponding permanent deletions to preserve problem~(1). They are properties of the complete validity rule, not of the secondary SSE criterion. With an arbitrary user rule, "**none**" is the reference unless the implications have been established.

Let $\mathcal{A}_{t-1}$ be the ordered candidate set at the beginning of endpoint t , and let $\mathcal{F}_t = \{s \in \mathcal{A}_{t-1} : V_\gamma(s, t) = 0\}$.

subtests	Permanent candidate action	Sufficient condition for equality with the unpruned recursion
"none"	Keep every boundary.	None.
"right"	Delete each currently invalid boundary.	Equation (3).
"both"	Delete the largest invalid boundary and every older boundary.	Equations (3) and (4).

subtests = "none": segment-count buckets

Here $\mathcal{A}_t = \{0, \dots, t\}$. The implementation groups existing boundaries by their already computed segment count,

$$\mathcal{B}_{t-1,k} = \{s < t : K_s = k\}, \quad k_t^* = \min\{k : \exists s \in \mathcal{B}_{t-1,k}, V_\gamma(s, t) = 1\}.$$

Equation~(2) immediately gives

$$K_t = k_t^* + 1, \quad Q_t = \min_{\substack{s \in \mathcal{B}_{t-1,k_t^*} \\ V_\gamma(s,t)=1}} \{Q_s + \mathcal{C}(s, t)\}.$$

The code therefore tests buckets in increasing k , stops after the first bucket containing a valid segment, and compares every valid SSE in that bucket. All larger- k buckets are irrelevant to both coordinates. Skipped incremental states remain stored and catch up if a later endpoint needs them. This is an exact lexicographic shortcut and does not assume that (K_t) is monotone. In particular, if boundary zero is valid, only that test is needed and $K_t = 1$.

subtests = "right": one-segment deferral and cost skipping

Outside its initial shortcut, right pruning applies

$$\mathcal{A}_t = \{s \in \mathcal{A}_{t-1} : V_\gamma(s, t) = 1\} \cup \{t\}.$$

While $V_\gamma(0, t) = 1$, boundary zero gives the unbeatable value $K_t = 1$. Later tests are deferred, although their candidates remain physically stored. When zero first fails, `replay_right_candidate()` reconstructs every deferred history and deletes a candidate if it failed at any missed endpoint. This is necessary: checking only its current endpoint would not reproduce right pruning when validity can recover.

Thereafter every survivor is tested and retained. During the ascending scan, once a provisional segment count K_t^{best} exists, the SSE is skipped for any survivor with $K_s + 1 > K_t^{\text{best}}$. Its validity must still be checked because it remains relevant to the future candidate set. A later candidate with a smaller K_s is still evaluated, so no monotonicity of (K_s) is assumed.

subtests = "both": stop at the newest failure

If $\mathcal{F}_t \neq \emptyset$, let $m_t = \max \mathcal{F}_t$. Inclusive-left pruning and right pruning together give

$$\mathcal{A}_t = \{s \in \mathcal{A}_{t-1} : s > m_t\} \cup \{t\}; \tag{5}$$

if there is no failure, all old boundaries survive. The optimized code scans from the newest boundary towards zero. At the first invalid boundary it stops: Equation~(5) deletes that candidate and every untested older candidate, so none can affect either the current optimum or a future candidate set. The surviving suffix is then processed in increasing boundary order. The code first finds its smallest $K_s + 1$, then computes SSE only for candidates in that segment-count class. The increasing pass preserves the previous smallest-boundary rule under an exact numerical tie.

The reverse stopping rule is an implementation shortcut, not an additional statistical pruning assumption. The statistical equality between "both" and "none" still relies on Equations~(3)–(4).

Operation counts and consequences

Let $A_t = |\mathcal{A}_{t-1}|$, the stored count reported by `nb[t]`, and let E_t be the number of candidate validity states actually checked. The candidate-loop work is controlled by $\sum_t E_t$, not necessarily by $\text{sum}(\text{nb}) = \sum_t A_t$.

For "none", $A_t = t$ and the stored exposure is exactly $n(n+1)/2$. If k_t^* is defined above, however,

$$E_t = \sum_{k < k_t^*} |\mathcal{B}_{t-1,k}| + |\mathcal{B}_{t-1,k_t^*}|.$$

For "right", the one-segment phase usually has $E_t = 1$ even though the physical vector grows; ending that phase incurs a one-off replay. Afterwards $E_t = A_t$, but SSE is computed only when the candidate can match or improve the provisional segment count. For "both", if a failure occurs then

$$E_t = |\{s \in \mathcal{A}_{t-1} : s \geq m_t\}|,$$

whereas $E_t = A_t$ when every candidate is valid. Hence "both" benefits most when a recent boundary fails; its reverse scan gives no saving at an endpoint with no failure.

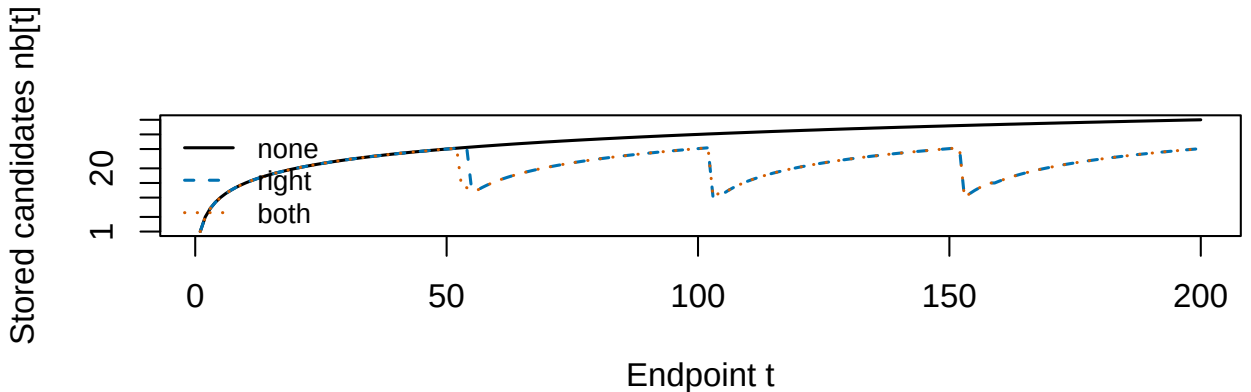
All three modes retain an $O(n^2)$ worst-case number of constant-time candidate checks. The shortcuts improve the realized count and constants, not that worst-case order. Test-specific work must also be included: Gaussian FOCuS maintains a functional state, while the exact AR(1) statistic can scan a whole candidate segment when requested and therefore has a larger unpruned worst case. Under the stability and regular-segmentation assumptions of the SVP paper (version 2), the non-decreasing sequence (K_t) and bounded spans of successive segment-count classes can instead yield the paper's near-linear regime. Those assumptions are not inferred merely from observing a small `nb` vector.

Simulated behavior

The examples use `ts_generator()` with Gaussian noise; no constant-valued series is used. First, consider four strong mean segments.

```
set.seed(20260914)
y_strong <- ts_generator(
  chpts = c(50, 100, 150, 200),
  parameters = c(0, 3, -2, 2),
  sd_noise = 1, type = "gauss"
)
gamma_strong <- 2 * log(length(y_strong))
strong_fits <- setNames(lapply(modes, function(mode) {
  SVP(y_strong, gamma_strong, test = "gaussian_mean", subtests = mode)
}), modes)
```

Mode	segment endpoints	K	sum(nb)	final candidates
none	50, 100, 150, 200	4	20100	201
right	50, 100, 150, 200	4	5655	52
both	50, 100, 150, 200	4	5564	52



The following noisy no-change series shows why pruning assumptions matter.

```
set.seed(13001)
y_recovery <- ts_generator(
  chpts = 30, parameters = 0, sd_noise = 1, type = "gauss"
)
recovery_fits <- setNames(lapply(modes, function(mode) {
  SVP(y_recovery, gamma = 2, test = "gaussian_mean", subtests = mode)
}), modes)
```

Mode	segment endpoints	K	sum(nb)
none	30	1	465
right	6, 30	2	256
both	7, 9, 10, 17, 30	5	193

Boundary zero is valid at $t = 30$, so **"none"** returns one segment. It failed at an earlier endpoint and was permanently deleted by **"right"**; **"both"** removes still more boundaries. This is a witness for the endpoint-only rule, not evidence that one statistical choice is universally preferable.

Comparison with the existing `time_gaussian` simulation

The version 0.2.0 benchmark reruns the exact checked-in design: 20 logarithmic sizes from 256 to 16,384 under no change (20 repetitions), and 0 to 100 changes at $n = 10,000$ (10 repetitions), for 5,640 paired fits. Every series is made with `ts_generator()` using the same seeds and is shared by all methods. The original `gaussian_time_*` files are preserved; new files use the `gaussian_time_shortcuts_*` prefix. Pairing uses experiment, n , k , repetition, and method.

Table 1: Median time at $n = 16,384$ with no change.

Method	Baseline (s)	Version 0.2.0 (s)	Speedup
PELT	0.208	0.210	0.99
SVP BIC	0.021	0.021	1.00
SVP BIC calibrated	0.021	0.021	1.00
SVP BIC multiscale	0.002	16.863	1.19e-04

The BIC and calibrated BIC methods use **"right"**; the multiscale method uses **"both"**. PELT is an unchanged timing control. The segment counts agree in 5566 of 5,640 rows. Every PELT and right-mode row agrees. The 74 disagreements are all multiscale no-change rows: the historical file was generated before the current `subtests` API and subsequent multiscale candidate corrections. A frozen build from immediately before the shortcut rewrite gives the same output as version 0.2.0 on a mismatching historical series, so these 74 changes must not be attributed to the new reverse scan.

The table shows that the two right-mode timings at the largest n are essentially unchanged. Thus this benchmark does not confirm a material speedup from skipping their relatively cheap SSE calculations; validity-state work dominates there. The old and new multiscale times are not an isolated shortcut comparison because their algorithmic outputs differ, and the reverse scan has no saving at an endpoint where every candidate is valid. Ratios are reported only when both medians are positive, since the timer records some short fits as zero. These are empirical wall-clock measurements, not complexity proofs. The complete before/after and speedup curves are saved in `simulations/time_gaussian/plots`.

Render environment. R version 4.6.1 (2026-06-24); `svpChange2` 0.2.0; aarch64-apple-darwin23.

Source basis. Mathematical notation follows the SVP paper, version 2. Implementation statements were checked against `src/svp.cpp` and `src/svp_internal.h` in `svpChange2` 0.2.0 on 14 September 2026.